

UNIT-III

IPC: Critical section problem:

- A **critical section** is a portion of code where a process accesses shared resources (e.g., files, memory).
- If multiple processes enter the critical section **simultaneously**, it may lead to **inconsistent data**.

Race Condition

- The situation where two or more processes try to access and manipulate the same data and output of the process depends on the orderly execution of those processes is called as Race Condition.
- count++ could be implemented as

```
register1 = count
register1 = register1 + 1
count = register1
```
- count-- could be implemented as

```
register2 = count
register2 = register2 - 1
count = register2
```
- Consider this execution interleaving with —count = 5 initially:
 - S0: producer execute register1 = count {register1 = 5}
 - S1: producer execute register1 = register1 + 1 {register1 = 6}
 - S2: consumer execute register2 = count {register2 = 5}
 - S3: consumer execute register2 = register2 - 1 {register2 = 4}
 - S4: producer execute count = register1 {count = 6}
 - S5: consumer execute count = register2 {count = 4}

Requirements for the Solution to Critical-Section Problem

1. **Mutual Exclusion:** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
 2. **Progress:** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
 3. **Bounded Waiting:** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.
- To general approaches are used to handle critical sections in operating systems: (1) Preemptive Kernel (2) Non Preemptive Kernel
 - Preemptive Kernel allows a process to be preempted while it is running in kernel mode.
 - Non Preemptive Kernel does not allow a process running in kernel mode to be preempted.
(these are free from race conditions)
 -

Peterson's Solution

- It is restricted to two processes that alternates the execution between their critical and remainder sections.
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted.
- The two processes share two variables:
 - int turn;
 - Boolean flag[2]
- The variable turn indicates whose turn it is to enter the critical section.
- The flag array is used to indicate if a process is ready to enter the critical section. flag[i] = true implies that process P_i is ready!

Note:- Peterson's Solution is a software based solution.

Algorithm for Process P_i

```
while (true) {  
    flag[i] = TRUE;  
    turn = j;  
    while ( flag[j] && turn == j);  
        CRITICAL SECTION  
    flag[i] = FALSE;  
        REMAINDER SECTION  
}
```

Fig: structure of process P_i in Peterson's solution

Solution to Critical Section Problem using Locks.

```
do{  
    acquire lock  
        Critical Section  
    release lock  
        Remainder Section  
}while (True);
```

Note:- Race Conditions are prevented by protecting the critical region by the locks.

Synchronization Hardware

- In general, we can provide any solution to critical section problem by using a simple tool called as LOCK where we can prevent the race condition.
- Many systems provide hardware support (hardware instructions available on several systems) for critical section code.

- In Uniprocessor hardware environment by disabling interrupts, we can solve the critical section problem. So that Currently running code would execute without any preemption.
- But by disabling interrupts on multiprocessor systems is time taking so that it is inefficient compared to Uniprocessor system.
- Now a days Modern machines provide special atomic hardware instructions that allow us to either *test memory word and set value* Or *swap contents of two memory words automatically* i.e. done through an uninterruptible unit.

Special Atomic hardware Instructions

- TestAndSet()
- Swap()
- The TestAndSet() Instruction is one kind of special atomic hardware instruction that allow us to test memory and set the value. We can provide Mutual Exclusion by using TestAndSet() instruction.

- Definition:

Boolean TestAndSet (Boolean *target)

```
{
    Boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

- Mutual Exclusion Implementation with TestAndSet()
- To implement Mutual Exclusion using TestAndSet() we need to declare Shared Boolean variable called as `_lock` (initialized to false) .

- **Solution:**

```
while (true) {
    while ( TestAndSet (&lock ))
        ; /* do nothing
        // critical section
    lock = FALSE;
    // remainder section
}
```

- The Swap() Instruction is another kind of special atomic hardware instruction that allow us to swap the contents of two memory words.
- By using Swap() Instruction we can provide Mutual Exclusion.

Definition: -

```
void Swap (Boolean *a, Boolean *b)
{
    Boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

- Shared Boolean variable called as `__lock` is to be declared to implement Mutual Exclusion in `Swap()` also, which is initialized to `FALSE`.

Solution:

```
while (true) {
    key = TRUE;
    while ( key == TRUE)
        Swap (&lock, &key);

    // critical section
    lock = FALSE;

    // remainder section
}
```

Note:- Each process has a local Boolean variable called as `__key`.

3. Mutex Locks (*Hardware-based Solution*)

- **Binary lock** (0 or 1) used for **mutual exclusion**.
- **Operations:**
 - `lock()` → Acquire lock before entering the critical section.
 - `unlock()` → Release lock after exiting.

Example: Mutex in C

```
pthread_mutex_t lock;
pthread_mutex_lock(&lock); // Acquire lock
// Critical Section
pthread_mutex_unlock(&lock); // Release lock
```

✗ Issue: Causes **busy waiting** (wastes CPU cycles).

Semaphores

- As it is difficult for the application programmer to use these hardware instructions, to overcome this difficulty we use the synchronization tool called as Semaphore (that does not require busy waiting)
- Semaphore S – integer variable, apart from this initialization we can access this only through two standard atomic operations called as wait() and signal().
- Originally the wait() and signal() operations are termed as P() and V() respectively. Which are termed from the Dutch words —proberen and —verhogen.
- *The definition for wait() is as follows:* wait

```
(S) {  
    while S <= 0  
        ; // no-op  
    S--;  
}
```

- *The definition for signal() is as follows:*

```
signal (S) {  
    S++;  
}
```

- All the modifications to the integer value of the semaphore in the wait() and signal() atomic operations must be executed indivisibly. i.e. when one process changes the semaphore value, no other process will change the same semaphore value simultaneously.
- *Usage of semaphore:-* we have two types of semaphores
 - Counting semaphore
 - Binary Semaphore.
- The value of the Counting Semaphore can range over an unrestricted domain.
- The value of the Binary Semaphore can range between 0 and 1 only.
- In some systems the Binary Semaphore is called as Mutex locks, because, as they are locks to provide the mutual exclusion.
- We can use the Binary Semaphore to deal with critical section problem for multiple processes.
- Counting Semaphores are used to control the access of given resource each of which consists of some finite no. of instances. This counting semaphore is initialized to number of resources available.
- The process that wish to use a resource must perform the wait() operation (count is decremented)
- The process that releases a resource must perform the signal() operation (count is incremented)
- When the count for the semaphore is 0 means that all the resources are being used by some processes. Otherwise resources are available for the processes to allocate .
- When a process is currently using a resource means that it blocks the resource until the count becomes > 0 .

- For example:
 - Let us assume that there are two processes p0 and p1 which consists of two statements s0 & s1 respectively.
 - Also assume that these two processes are running concurrently such that process p1 executes the statement s1 only after process p0 executes the statement s0.
 - Let us assume the process p0 & p1 share the same semaphore called as —synchl which is initialized to 0 by inserting the statements

S0;

Signal (synch); in

process p0 and the statements

wait (synch);

S1;

in process p1 .

Implementation:

- The main disadvantage of the semaphore definition is, it requires the busy waiting.
- Because when one process is in critical section and if another process needs to enter in to the critical section must have to loop in the entry code continuously.

Implementation of semaphore with no busy waiting:

- To overcome the need of the busy waiting we have to modify the definition of wait() and signal() operations. i.e. when a process executes wait() operation and finds that it is not positive then it must wait.
- Instead of engaging the busy wait, the process block itself so that there will be a chance to the CPU to select another process for execution. It is done by block() operation.
 - Blocked processes are placed in waiting queue.
- Later the process that has already been blocked by itself is restarted by using wakeup() operation, so that the process will move from waiting state to ready state.
 - Blocked processes that are placed in waiting queue are now placed into ready queue.
- To implement the semaphore with no busy waiting we need to define the semaphore of the wait() and signal() operation by using the __C' Struct. Which is as follows:

typedef struct {

int value;

struct process *list;

}semaphore;

- i.e. each semaphore has an integer value stored in the variable —value1 and the list of processes list.

- When a process perform the wait() operation on the semaphore then it will adds list of processes to the list .
- When a process perform the signal() operation on the semaphore then it removes the processes from the list.

Semaphore Implementation with no Busy waiting Implementation of

wait: (definition of wait with no busy waiting)

```
wait (S){
    value--;
    if (value < 0) {
        add this process to waiting queue
        block(); }
}
```

Implementation of signal: (definition of signal with no busy waiting) Signal (S){

```
value++;
if (value <= 0) {
    remove a process P from the waiting queue
    wakeup(P); }
}
```

Deadlock and Starvation

- The implementation of semaphore with waiting queue may result in the situation where two or more processes are waiting for an event is called as Deadlocked.
- To illustrate this, let us assume two processes P0 and P1 each accessing two semaphores S and Q which are initialized to 1 :-

<i>P0</i>	<i>P1</i>
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

- Now process P0 executes *wait(S)* and P1 executes *wait(Q)*, assume that P0 wants to execute *wait(Q)* and P1 executes *wait(S)*. But it is possible only after process P1 executes the *signal(Q)* and P0 executes *signal(S)*.
- Starvation or indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended.
-

Classical Problems of Synchronization

- **Bounded-Buffer Problem**
- **Readers and Writers Problem**
- **Dining-Philosophers Problem**

Bounded-Buffer Problem

- Let us assume N buffers, each can hold only one item.
- Semaphore mutex initialized to the value 1 which is used to provide mutual exclusion.
- Semaphore full initialized to the value 0
- Semaphore empty initialized to the value N .
- Semaphore full and empty are used to count the number of buffers.
- The structure of the producer process

```
while (true) {  
    // produce an item  
    wait (empty);  
    wait (mutex);  
    // add the item to the buffer signal  
    (mutex);  
    signal (full);  
}
```

The structure of the consumer process while

```
(true) {  
    wait (full);  
    wait (mutex);  
    // remove an item from buffer signal  
    (mutex);  
    signal (empty);  
    // consume the removed item  
}
```

Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - Readers – only read the data set, do not perform any updates
 - Writers – can both read and write the data set (perform the updates).
- If two readers read the shared data simultaneously, there will be no problem. If both a reader(s) and writer share the same data simultaneously then there will be a problem.

- In the solution of reader-writer problem, the reader process share the following data structures:

Semaphore Mutex, wrt; int

readcount;

- Where ☐ Semaphore mutex is initialized to 1.
- ☐ Semaphore wrt is initialized to 1.
- ☐ Integer readcount is initialized to 0.

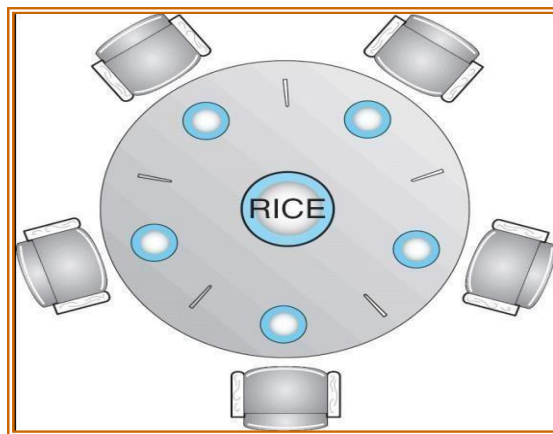
The structure of a writer process

```
while (true) {
    wait (wrt) ;
    // writing is performed
    signal (wrt) ;
}
```

The structure of a reader process

```
while (true) {
    wait (mutex) ;
    readcount ++ ;
    if (readcount == 1) wait (wrt) ;
    signal (mutex)
    // reading is performed wait
    (mutex) ;
    readcount - - ;
    if (readcount == 0) signal (wrt) ; signal
    (mutex) ;
}
```

Dining-Philosophers Problem



-
- **Shared data**
 - Bowl of rice (data set)
 - Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem

- **The structure of Philosopher i:**

```
While (true) {
    wait ( chopstick[i] );
    wait ( chopstick[ (i + 1) % 5] );
    // eat
    signal ( chopstick[i] );
    signal ( chopstick[ (i + 1) % 5] );
    // think      }
```

Problems with Semaphores

- Incorrect use of semaphore operations:
 - signal (mutex) wait (mutex) ☐ Case 1
 - wait (mutex) ... wait (mutex) ☐ Case 2
 - Omitting of wait (mutex) or signal (mutex) (or both) ☐ Case 3
- As the semaphores used incorrectly as above may results the timing errors.
- Case 1 ☐ Several processes may execute in critical section by violating the mutual exclusion requirement.
- Case 2 ☐ Dead lock will occur.
- Case 3 ☐ either mutual exclusion is violated or dead lock will occur
- To deal with such type of errors, researchers have developed high-level language constructs.
- One type of high-level language constructs that is to be used to deal with the above type of errors is ☐ the *Monitor* type.

Monitors

- A high-level abstraction that provides a convenient and effective mechanism for process synchronization.
- A procedure can access only those variables that are declared in a monitor and formal parameters .
- Only one process may be active within the monitor at a time

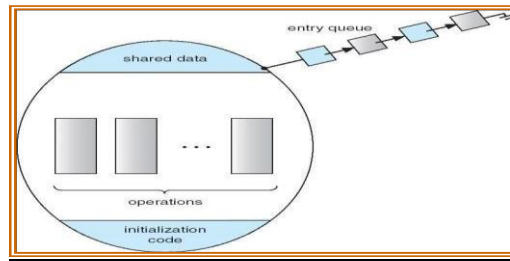
Syntax of the monitor :-

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }
    ...
```

procedure Pn (...) {.....}

Initialization code (....) { ... }

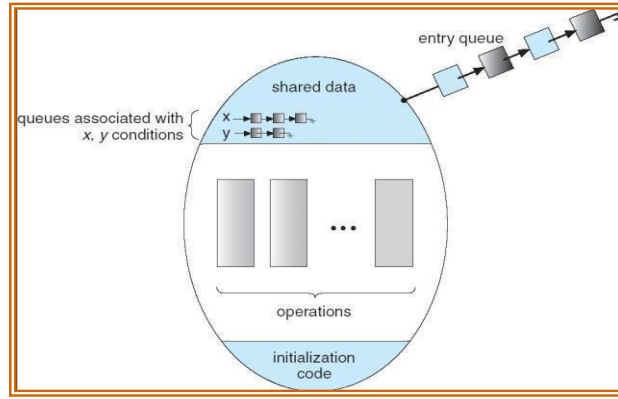
Schematic view of a Monitor



Condition Variables

- Synchronization scheme is not effective with in the monitors.
- A programmer who needs to write the synchronization scheme can define one or more variables of type *Condition*
- condition x, y;
- The only operations that can be invoked on a condition variable are wait() and signal().
- The operations are
 - x.wait () – a process that request an operation is suspended until another process invokes x.signal ()
 - x.signal () – resumes only one suspended processes (if any) that invoked x.wait ()
- Now suppose that when x.signal() operation is invoked by a process P, there is a suspended process Q associated with condition x. if Q is allowed to resume its execution, the signaling process P must wait. Else both P and Q would be active simultaneously with in a monitor.
- There are two possibilities
 - 1) signal and wait:- P either waits until Q leaves the monitor or waits for another condition.
 - 2) signal and continue:- Q either waits Until P leaves the monitor or waits for another condition.

Monitor with Condition Variables



Solution to Dining Philosophers using Monitors

```
monitor DP
{
    enum { THINKING, HUNGRY, EATING } state [5];
    condition self [5];
    void pickup (int i) {
        state[i] = HUNGRY; test(i);
        if (state[i] != EATING) self [i].wait;
        void putdown (int i) {
            state[i] = THINKING;
            // test left and right neighbors test((i
            + 4) % 5);
            test((i + 1) % 5);
        }
    }
    void test (int i) {
        if ( (state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING) ) {

            state[i] = EATING ; self[i].signal
            ();
        }
    }
    initialization_code() {
        for (int i = 0; i < 5; i++)
            state[i] = THINKING;
    }
}
```

Each philosopher i invokes the operations `pickup()` and `putdown()` in the following sequence:

```
dp.pickup (i)
    EAT
dp.putdown (i)
```

Monitor Implementation Using Semaphores Variables

```
semaphore mutex; // (initially = 1) semaphore
next;           // (initially = 0)
int next-count = 0; Each
```

procedure F will be replaced by

```
wait(mutex);
...
body of  $F$ ;
...
if (next-count > 0)
    signal(next)
else
    signal(mutex);
```

Mutual exclusion within a monitor is ensured.

Monitor Implementation

For each condition variable x , we have:

```
semaphore x-sem; // (initially = 0) int
x-count = 0;
```

The operation x .wait can be implemented as:

```
x-count++;
if (next-count > 0)
    signal(next);
else
    signal(mutex);
wait(x-sem);
```

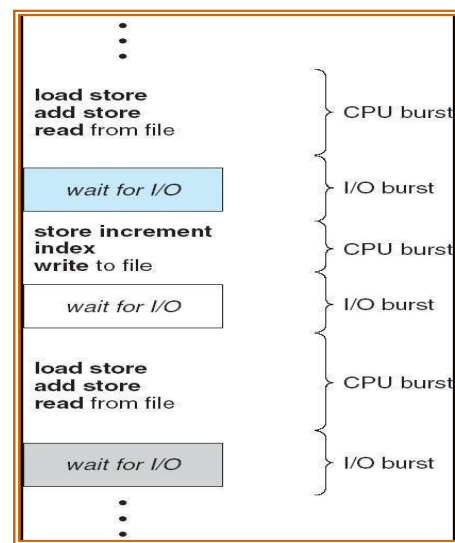
```
x-count--;
```

The operation x .signal can be implemented as:

```
if (x-count > 0) {
    next-count++;
    signal(x-sem);
    wait(next);
    next-count--;
}
```

Synchronization Examples

- Windows XP
- Linux



Windows XP Synchronization

- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses spinlocks on multiprocessor systems
- Also provides dispatcher objects which may act as either mutexes and semaphores
- Dispatcher objects may also provide events
 - An event acts much like a condition variable

Linux Synchronization

- Linux:
 - disables interrupts to implement short critical sections
- Linux provides:
 - semaphores
 - spin locks

CPU Scheduling

- ☐ Maximum CPU utilization obtained with multiprogramming
- ☐ CPU-I/O Burst Cycle – Process execution consists of a *cycle* of CPU execution and I/O wait
- ☐ CPU burst distribution
- ☐ Alternating Sequence of CPU & I/O Bursts

CPU Scheduler

- ☐ Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them
- ☐ CPU scheduling decisions may take place when a process:
 2. Switches from running to waiting state
 3. Switches from running to ready state
 4. Switches from waiting to ready
 5. Terminates

Scheduling under 1 and 4 is *nonpreemptive*

All other scheduling is *preemptive*

Dispatcher

Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:

switching context

switching to user mode

jumping to the proper location in the user program to restart that program *Dispatch* ☐

latency – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria

CPU utilization – keep the CPU as busy as possible

Throughput – No. of processes that complete their execution per time unit

Turnaround time – amount of time to execute a particular process

Waiting time – amount of time a process has been waiting in the ready queue

Response time – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Optimization Criteria

- ☐ Max CPU utilization
- ☐ Max throughput Min
- ☐ turnaround time
- ☐ Min waiting time
- ☐ Min response time

Deadlock:

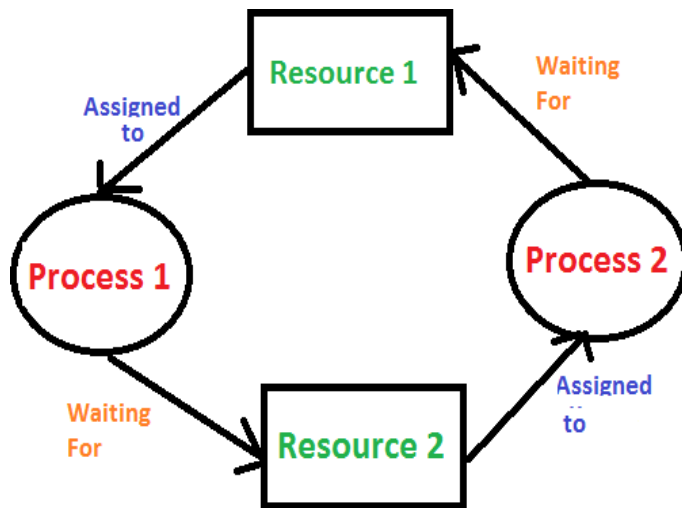
Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Consider an example when two trains are coming toward each other on same track and there is only one track, none of the trains can move once they are in front of each other. Similar situation occurs in operating systems when there are two or more processes hold some resources and wait for resources held by other(s).

For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.

Under the normal mode of operation, a process may utilize a resource in only the following sequence:

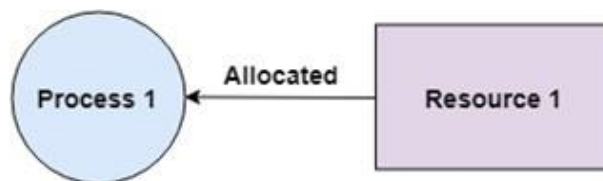
- **Request:** If the request cannot be granted immediately, then the Requesting process must wait until it can acquire the resource.
- **Use:** The process can operate on the resource.
- **Release:** The process releases the resource.



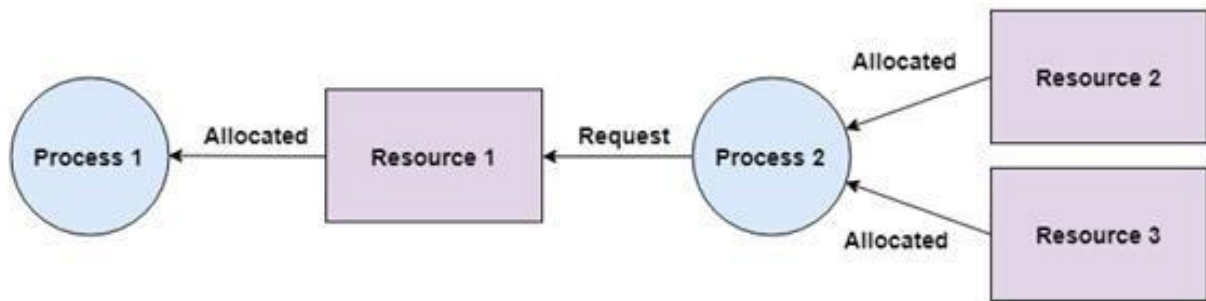
Deadlock Characterization:

In a deadlock process never finish executing and system resources are tied up. A deadlock situation can arise if the following four conditions hold simultaneously in a system.

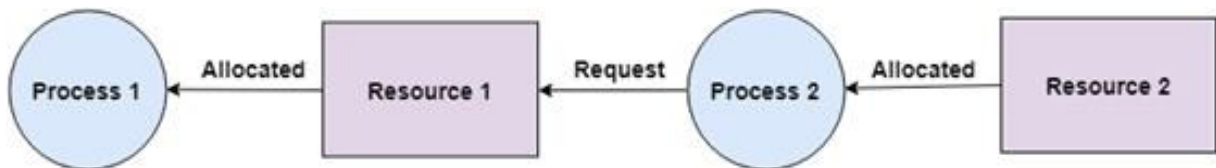
a) Mutual Exclusion: There should be a resource that can only be held by one process at a time. In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



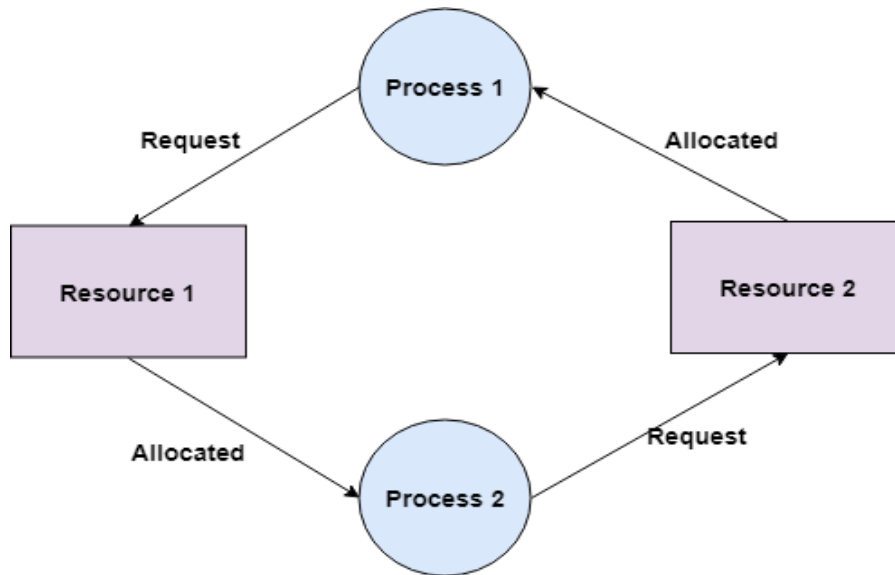
b) Hold and wait: A process can hold multiple resources and still request more resources from other processes which are holding them. In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



c) No Pre-emption: A resource cannot be pre-empted from a process by force. A process can only release a resource voluntarily. In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 surrenders it voluntarily after its execution is complete.



d) Circular Wait: A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process. This forms a circular chain. For example: Process 1 is allocated Resource 2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2. This forms a circular wait loop.

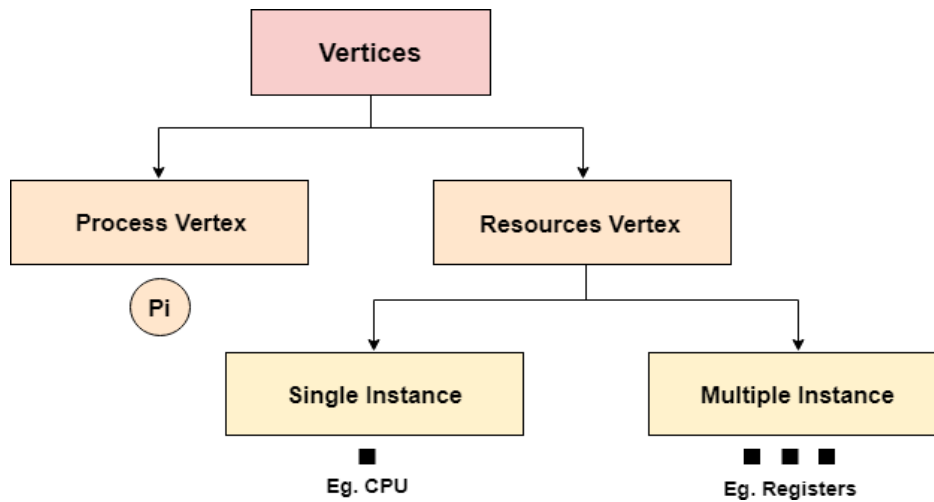


Resource Allocation Graph:

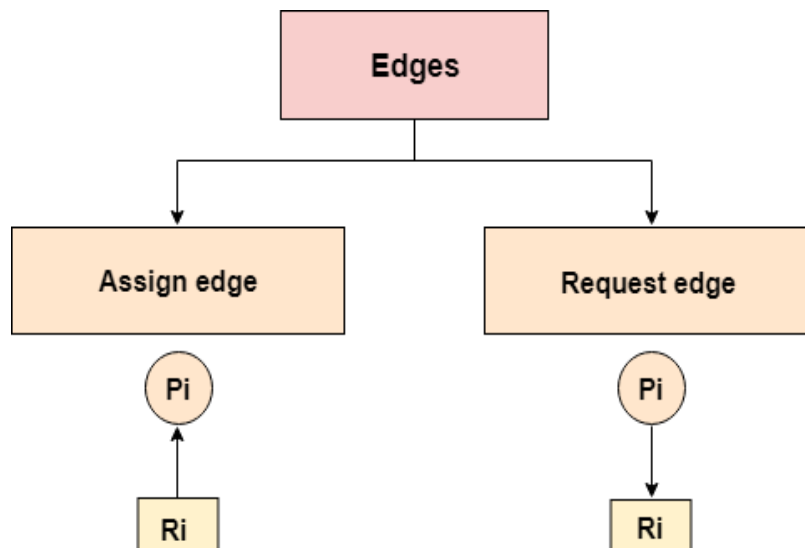
The **resource allocation graph** is the pictorial representation of the state of a system. As its name suggests, the resource allocation graph is the complete information about all the processes which are holding some resources or waiting for some resources. It also contains the information about all the instances of all the resources whether they are available or being used by the processes.

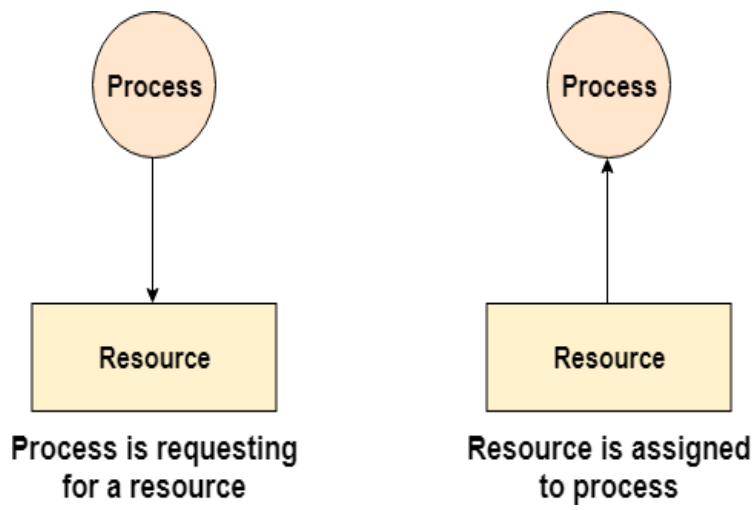
In Resource allocation graph, the process is represented by a Circle while the Resource is represented by a rectangle. Let's see the types of vertices and edges in detail.

Vertices are mainly of two types, **Resource** and **process**. Each of them will be represented by a different shape. Circle represents process while rectangle represents resource. A resource can have more than one instance. Each instance will be represented by a dot inside the rectangle.



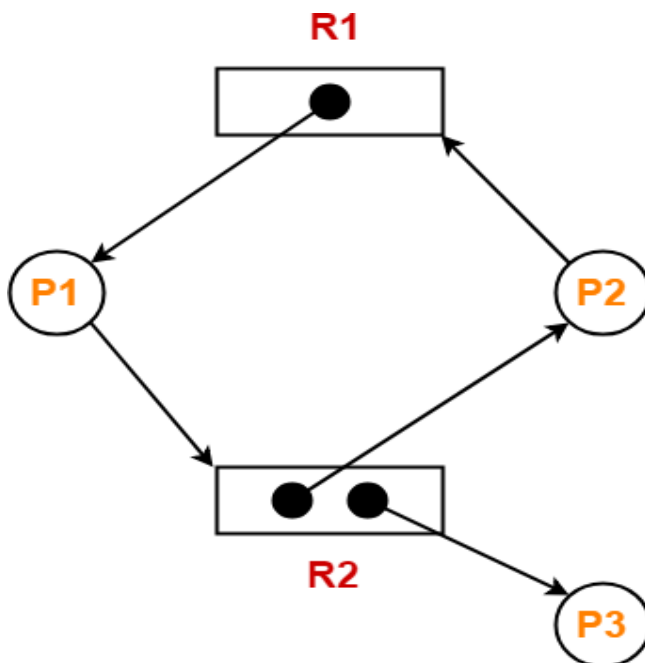
- Edges in RAG are also of two types, one represents **assignment** and other represents the **wait of a process for a resource**. The above image shows each of them.





Example: There exist three processes in the system namely P1, P2 and P3.

There exist two resources in the system namely R1 and R2. There exists a single instance of resource R1 and two instances of resource R2. Process P1 holds one instance of resource R1 and is waiting for an instance of resource R2. Process P2 holds one instance of resource R2 and is waiting for an instance of resource R1. Process P3 holds one instance of resource R2 and is not waiting for anything.



Deadlock handling methods

The various strategies for handling deadlock are-

- **Deadlock Prevention**
- **Deadlock Avoidance**
- **Deadlock Detection and Recovery**

I) **Deadlock Prevention:**

a) **Eliminate Hold and wait :**

- Allocate all required resources to the process before the start of its execution, this way hold and wait condition is eliminated but it will lead to low device utilization. For example, if a process requires printer at a later time and we have allocated printer before the start of its execution printer will remain blocked till it has completed its execution.
- The process will make a new request for resources after releasing the current set of resources. This solution may lead to starvation.

b) **Eliminate No Preemption:** Preempt resources from the process when resources required by other high priority processes.

c) **Eliminate Circular Wait:** Each resource will be assigned with a numerical number. A process can request the resources only in increasing order of numbering.

d) For Example, if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted.

ii) **Deadlock Avoidance:** One of the most commonly used deadlock avoidance algorithms is the *Banker's algorithm* introduced by Dr.D.W. Dijkstra in 1965.

- It uses similar concept used by banker to decide if the loan can be granted or not.

Banker's Algorithm: Bankers' Algorithm is resource allocation and deadlock avoidance algorithm which test the entire request made by processes for resources, it checks for the **safe state**, if after granting request system remains in the safe state it allows the request and if there is **no safe state** it doesn't allow the request made by the process.

Inputs to Banker's Algorithm:

- Max need of resources by each process.

- Currently allocated resources by each process.
- Max free available resources in the system.

The request will only be granted under the below condition:

- If the request made by the process is less than equal to max need to that process.
- If the request made by the process is less than equal to the freely available resource in the system.

Bankers algorithm(or)Safety algorithm:

- $work[1..m] = available$ /* how many resources are available */
- $finish[1..n] = false$ (for all i) /* none finished yet */

Step1: Find an i such that **finish[i]=false** and **need[i] <= work** /*find a proc that can complete is request now

If no such i exists, go to step 3 /* we are done*/

Step 2: Found an i: Finish [i] = true /* done with this process */

work = work + allocation [i] /* assume this process were to finish, and its allocation back to the available list */

Go to step 1

Step 3: If finish[i] = true for all i, the system is safe. Else

Not

Example:

Total Resources	R1	R2	R3
	10	5	7

Process	Allocation			Max		
	R1	R2	R3	R1	R2	R3
P1	0	1	0	7	5	3
P2	2	0	0	3	2	2
P3	3	0	2	9	0	2
P4	2	1	1	2	2	2

Total resources are $R1 = 10$, $R2 = 5$, $R3 = 7$ and allocated resources are

- $R1 = (0+2+3+2) = 7$
- $R2 = (1+0+0+1) = 2$,
- $R3 = (0+0+2+1) = 3$.

Remaining available = Total resources – allocated resources Therefore, remaining resources are

- $R1 = (10 - 7) = 3$,
- $R2 = (5 - 2) = 3$,
- $R3 = (7 - 3) = 4$.

Remaining need = max – allocated

Process	Max			Allocation			Available			Needed		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	7	5	3	0	1	0	3	3	4	7	4	3
P2	3	2	2	2	0	0				1	2	2
P3	9	0	2	3	0	2				6	0	0
P4	2	2	2	2	1	1				0	1	1
				7	2	3						

Now we can check **need** ≤ **work available**

For $p1 \Rightarrow 7\ 4\ 3 \leq 3\ 3\ 4$, here condition not satisfied

For $p2 \Rightarrow 1\ 2\ 2 \leq 3\ 3\ 4$, Here condition Satisfied . then

available=Available+allocation

$$3\ 3\ 4 + 2\ 0\ 0 = 5\ 3\ 4$$

For $p3 \Rightarrow 6\ 0\ 0 \leq 5\ 3\ 4$, here condition not satisfied

For $p4 \Rightarrow 0\ 0\ 1 \leq 5\ 3\ 4$, Here Condition satisfied

then **available=Available+ allocation.**

$$5\ 3\ 4 + 2\ 1\ 1 = 7\ 4\ 5$$

For $p1 = 7\ 4\ 3 \leq 7\ 4\ 5$, Here condition Satisfied . then

available=Available+allocation

$$7\ 4\ 5 + 0\ 1\ 0 = 7\ 5\ 5$$

For $p3 = 6\ 0\ 0 \leq 7\ 5\ 5$, Here condition Satisfied . then

available=Available+allocation

$$7\ 5\ 5 + 3\ 0\ 2 = 10\ 5\ 7$$

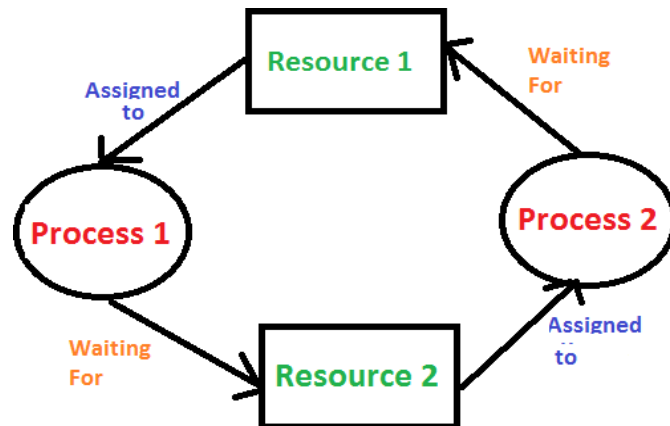
The Final safe sequence for avoiding deadlock for process is

<p2,p4,p1,p3>

Deadlock Detection And Recovery :

Deadlock Detection:

1. If the Resources have Single Instances : In this case for Deadlock detection we can run an algorithm to check for cycle in the Resource Allocation Graph. Presence of cycle in the graph is the sufficient condition for deadlock.



- In the above diagram, resource 1 and resource 2 have single instances. There is a cycle $R1 \rightarrow P1 \rightarrow R2 \rightarrow P2$. So, Deadlock is Confirmed.

2) If there are multiple instances of resources: Detection of the cycle is necessary but not sufficient condition for deadlock detection, in this case, the system may or may not be in deadlock varies according to different situations.

Deadlock Recovery:

A traditional operating system such as Windows doesn't deal with deadlock recovery as it is time and space consuming process. Real-time operating systems use Deadlock recovery.

Recovery method:

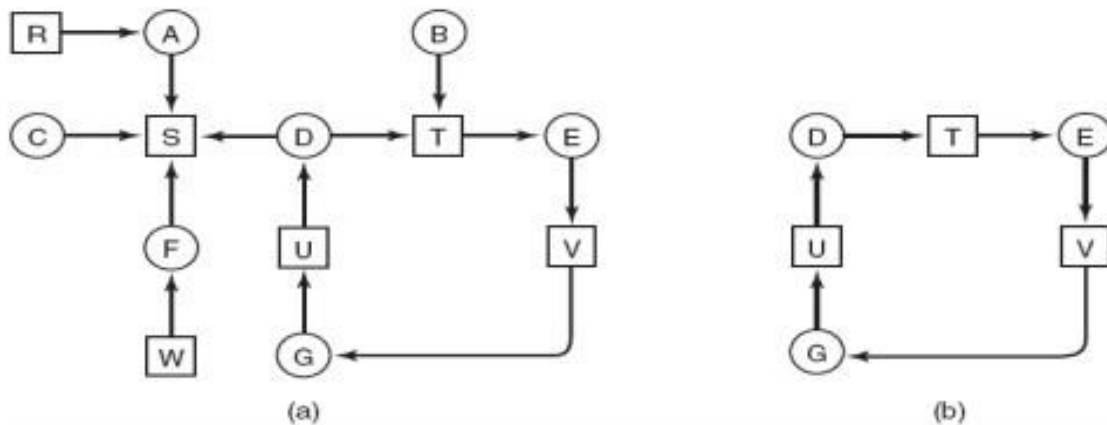
1. Killing the process: killing all the process involved in the deadlock. Killing process one by one. After killing each process check for deadlock again keep repeating the process till system recover from deadlock.

2) Resource Pre-emption: Resources are pre-empted from the processes involved in the deadlock, pre-empted resources are allocated to other processes so that there is a possibility of recovering the system from deadlock. In this case, the system goes into starvation.

Deadlock detection with single resource of each type.

Algorithm for detecting deadlock for single resource

1. For each node, N in the graph, perform the following five steps with N as the starting node.
2. Initialize L to the empty list, designate all arcs as unmarked.
3. Add current node to end of L, check to see if node now appears in L two times. If it does, graph contains a cycle (listed in L), algorithm terminates.
4. From given node, see if any unmarked outgoing arcs. If so, go to step 5; if not, go to step 6.
5. Pick an unmarked outgoing arc at random and mark it. Then follow it to the new current node and go to step 3.
6. If this is initial node, graph does not contain any cycles, algorithm terminates. Otherwise, dead end. Remove it, go back to previous node, make that one current node, go to step 3.



(a) A resource graph. (b) A cycle extracted from (a).